BinarySearch Report

Overall Score

23.828161

Matched Submissions

MergeSort_Plagiarized1 (3.0)	BinarySearchTree_Plagiarized1 (5.0)	MinHeap (3.0)	
BinarySearch_Plagiarized1 (24.0)	PrimeNumbers (3.0)	DijkstraAlgorithm (3.0)	Queue (3.0)
Fibonacci (3.0)	Fibonacci_Plagiarized1 (8.0)	QuickSort_Plagiarized1 (3.0)	FinancialCalculator (3.0)
Graph (3.0)	StringUtils (5.0)	Graph_Plagiarized1 (3.0)	StringUtils_Plagiarized1 (3.0)
HashMapImpl (3.0)	HashMapImpl_Plagiarized1 (3.0)	LinkedList (3.0)	LinkedList_Plagiarized1 (3.0)
LRUCache (3.0)	MatrixOperations (19.0)	MatrixOperations_Plagiarized1 (3.0)	BinarySearchTree (5.0)

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

MergeSort_Plagiarized1: MergeSort_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Merge sort algorithm implementation
5 |  */
6 | public class MergeSort_Plagiarized1 {
7 |
8 |     public void performSort(int[] arr) {
9 |         if (arr == null || arr.length <= 1) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

Graph: Graph.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Graph representation using adjacency list
5 |  */
6 | import java.util.*;
7 |
8 | public class Graph {
9 |     private int vertices;
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

Queue: Queue.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Queue implementation using circular array
5 |  */
6 | public class Queue {
7 |     private static final int DEFAULT_CAPACITY = 10;
8 |     private int[] elements;
9 |     private int front;
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

QuickSort_Plagiarized1: QuickSort_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Sorting algorithm using quick sort
5 |  */
6 | public class QuickSort_Plagiarized1 {
7 |
8 |     public void performSort(int[] arr) {
9 |         if (arr == null || arr.length == 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

MatrixOperations: MatrixOperations.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Matrix operations utility class
5 |  */
6 | public class MatrixOperations {
7 |
8 |     public int[][] add(int[][] matrix1, int[][] matr
  | ix2) {
9 |         if (!canOperate(matrix1, matrix2)) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search - renamed version
5 |  */
6 | public class BinarySearch_Plagiarized1 {
7 |
8 |     public int find(int[] arr, int value) {
9 |         if (arr == null || arr.length == 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

BinarySearchTree: BinarySearchTree.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * A simple Binary Search Tree implementation
5 |  */
6 | public class BinarySearchTree {
7 |     private Node root;
8 |
9 |     private class Node {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

PrimeNumbers: PrimeNumbers.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Prime number utilities
5 |  */
6 | public class PrimeNumbers {
7 |
8 |     public boolean isPrime(int n) {
9 |         if (n <= 1) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

LRUCache: LRUCache.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * LRU Cache implementation using HashMap and Doubly Linked List
5 |  */
6 | import java.util.*;
7 |
8 | public class LRUCache {
9 |     private final int capacity;
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

DijkstraAlgorithm: DijkstraAlgorithm.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Dijkstra's shortest path algorithm
5 |  */
6 | import java.util.*;
7 |
8 | public class DijkstraAlgorithm {
9 |
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

HashMapImpl: HashMapImpl.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Simple HashMap implementation using separate chaining
5 |  */
6 | public class HashMapImpl {
7 |     private static final int DEFAULT_CAPACITY = 16;
8 |     private static final double LOAD_FACTOR = 0.75;
9 |
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

Fibonacci: Fibonacci.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Fibonacci number calculator
5 |  */
6 | public class Fibonacci {
7 |
8 |     public long calculate(int n) {
9 |         if (n < 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

MatrixOperations_Plagiarized1: MatrixOperations_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Matrix math operations
5 |  */
6 | public class MatrixOperations_Plagiarized1 {
7 |
8 |     public int[][] addMatrices(int[][] m1, int[][] m2) {
9 |         if (!dimensionsMatch(m1, m2)) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

Fibonacci_Plagiarized1: Fibonacci_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Calculating Fibonacci numbers
5 |  */
6 | public class Fibonacci_Plagiarized1 {
7 |
8 |     public long compute(int num) {
9 |         if (num < 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

HashMapImpl_Plagiarized1: HashMapImpl_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * HashMap with chaining collision resolution
5 |  */
6 | public class HashMapImpl_Plagiarized1 {
7 |     private static final int INITIAL_SIZE = 16;
8 |     private static final double MAX_LOAD = 0.75;
9 |
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

LinkedList_Plagiarized1: LinkedList_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Singly linked list data structure
5 |  */
6 | public class LinkedList_Plagiarized1 {
7 |     private Element head;
8 |     private int length;
9 |
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

BinarySearchTree_Plagiarized1: BinarySearchTree_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search Tree - slightly modified version
5 |  */
6 | public class BinarySearchTree_Plagiarized1 {
7 |     private TreeNode root;
8 |
9 |     private class TreeNode {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

MinHeap: MinHeap.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Heap data structure implementation
5 |  */
6 | public class MinHeap {
7 |     private int[] heap;
8 |     private int size;
9 |     private int capacity;
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

FinancialCalculator: FinancialCalculator.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Calculator for working with currency and financial calculations
5 |  */
6 | public class FinancialCalculator {
7 |
8 |     public double calculateSimpleInterest(double principal, double rate, double time) {
9 |         if (principal < 0 || rate < 0 || time < 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

Graph_Plagiarized1: Graph_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Graph using adjacency list - modified
5 |  */
6 | import java.util.*;
7 |
8 | public class Graph_Plagiarized1 {
9 |     private int nodeCount;
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

StringUtils_Plagiarized1: StringUtils_Plagiarized1.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * String helper methods
5 |  */
6 | public class StringUtils_Plagiarized1 {
7 |
8 |     public boolean checkPalindrome(String text) {
9 |         if (text == null || text.length() == 0) {
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

LinkedList: LinkedList.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * A simple singly linked list implementation
5 |  */
6 | public class LinkedList {
7 |     private Node head;
8 |     private int size;
9 |
```

Match #0: N-Gram Match Group

Similarity: 84.8%

BinarySearch: BinarySearch.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
```

StringUtils: StringUtils.java (Lines 1–9)

```
1 | package testdata;
2 |
3 | /**
4 |  * String manipulation utilities
5 |  */
6 | public class StringUtils {
7 |
8 |     public boolean isPalindrome(String str) {
9 |         if (str == null || str.length() == 0) {
```

Match #1: N-Gram Match Group

Similarity: 81.8%

BinarySearch: BinarySearch.java (Lines 68–82)

```
68 |                 right = mid - 1;
69 |             } else if (array[mid] > target) {
70 |                 right = mid - 1;
71 |             } else {
72 |                 left = mid + 1;
73 |             }
74 |         }
75 |
76 |         return result;
77 |     }
78 |
79 |     public int findLast(int[] array, int target) {
80 |         int result = -1;
81 |         int left = 0;
82 |         int right = array.length - 1;
```

Fibonacci_Plagiarized1: Fibonacci_Plagiarized1.java (Lines 45–60)

```
45 |
46 |         long[] result = new long[length];
47 |
48 |         for (int idx = 0; idx < length; idx++) {
49 |             result[idx] = computeIterative(idx);
50 |         }
51 |
52 |         return result;
53 |     }
54 |
55 |     public long computeWithCache(int num) {
56 |         if (num < 0) {
57 |             throw new IllegalArgumentException("n must be non-negative");
58 |         }
59 |
60 |         long[] cache = new long[num + 1];
```

Match #1: N-Gram Match Group

Similarity: 81.8%

BinarySearch: BinarySearch.java (Lines 68–82)

```
68 |                 right = mid - 1;
69 |             } else if (array[mid] > target) {
70 |                 right = mid - 1;
71 |             } else {
72 |                 left = mid + 1;
73 |             }
74 |         }
75 |
76 |         return result;
77 |     }
78 |
79 |     public int findLast(int[] array, int target) {
80 |         int result = -1;
81 |         int left = 0;
82 |         int right = array.length - 1;
```

MatrixOperations: MatrixOperations.java (Lines 15–29)

```
15 |         int[][] result = new int[rows][cols];
16 |
17 |         for (int i = 0; i < rows; i++) {
18 |             for (int j = 0; j < cols; j++) {
19 |                 result[i][j] = matrix1[i][j] + matrix2[i][j];
20 |             }
21 |         }
22 |
23 |         return result;
24 |     }
25 |
26 |     public int[][] subtract(int[][] matrix1, int[][] matrix2) {
27 |         if (!canOperate(matrix1, matrix2)) {
28 |             throw new IllegalArgumentException("Matrices must have same dimensions");
29 |         }
```

Match #2: N-Gram Match Group

Similarity: 83.5%

BinarySearch: BinarySearch.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
10 |             return -1;
11 |         }
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search - renamed version
5 |  */
6 | public class BinarySearch_Plagiarized1 {
7 |
8 |     public int find(int[] arr, int value) {
9 |         if (arr == null || arr.length == 0) {
10 |             return -1;
11 |         }
```

Match #2: N-Gram Match Group

Similarity: 83.5%

BinarySearch: BinarySearch.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
10 |             return -1;
11 |         }
```

BinarySearchTree: BinarySearchTree.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * A simple Binary Search Tree implementation
5 |  */
6 | public class BinarySearchTree {
7 |     private Node root;
8 |
9 |     private class Node {
10 |         int data;
11 |         Node left;
```

Match #2: N-Gram Match Group

Similarity: 83.5%

BinarySearch: BinarySearch.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search implementation
5 |  */
6 | public class BinarySearch {
7 |
8 |     public int search(int[] array, int target) {
9 |         if (array == null || array.length == 0) {
10 |             return -1;
11 |         }
```

BinarySearchTree_Plagiarized1: BinarySearchTree_Plagiarized1.java (Lines 1–11)

```
1 | package testdata;
2 |
3 | /**
4 |  * Binary Search Tree - slightly modified version
5 |  */
6 | public class BinarySearchTree_Plagiarized1 {
7 |     private TreeNode root;
8 |
9 |     private class TreeNode {
10 |         int value;
11 |         TreeNode leftChild;
```

Match #3: N-Gram Match Group

Similarity: 86.7%

BinarySearch: BinarySearch.java (Lines 68–84)

```
68 | 68 |                 right = mid - 1;
69 | 69 |             } else if (array[mid] > target) {
70 | 70 |                 right = mid - 1;
71 | 71 |             } else {
72 | 72 |                 left = mid + 1;
73 | 73 |             }
74 | 74 |         }
75 | 75 |
76 | 76 |         return result;
77 | 77 |     }
78 | 78 |
79 | 79 |     public int findLast(int[] array, int target) {
80 | 80 |         int result = -1;
81 | 81 |         int left = 0;
82 | 82 |         int right = array.length - 1;
83 | 83 |
84 | 84 |         while (left <= right) {
```

MatrixOperations: MatrixOperations.java (Lines 15–31)

```
15 | 15 |         int[][] result = new int[rows][cols];
16 | 16 |
17 | 17 |         for (int i = 0; i < rows; i++) {
18 | 18 |             for (int j = 0; j < cols; j++) {
19 | 19 |                 result[i][j] = matrix1[i][j] + matrix2[i][j];
20 | 20 |             }
21 | 21 |         }
22 | 22 |
23 | 23 |         return result;
24 | 24 |     }
25 | 25 |
26 | 26 |     public int[][] subtract(int[][] matrix1, int[][] matrix2) {
27 | 27 |         if (!canOperate(matrix1, matrix2)) {
28 | 28 |             throw new IllegalArgumentException("Matrices must have same dimensions");
29 | 29 |         }
30 | 30 |
31 | 31 |         int rows = matrix1.length;
```

Match #4: N-Gram Match Group

Similarity: 83.3%

BinarySearch: BinarySearch.java (Lines 89–99)

```
89 | 89 |                 left = mid + 1;
90 | 90 |             } else if (array[mid] > target) {
91 | 91 |                 right = mid - 1;
92 | 92 |             } else {
93 | 93 |                 left = mid + 1;
94 | 94 |             }
95 | 95 |         }
96 | 96 |
97 | 97 |         return result;
98 | 98 |     }
99 | 99 | }
```

MatrixOperations: MatrixOperations.java (Lines 33–49)

```
33 | 33 |         int[][] result = new int[rows][cols];
34 | 34 |
35 | 35 |         for (int i = 0; i < rows; i++) {
36 | 36 |             for (int j = 0; j < cols; j++) {
37 | 37 |                 result[i][j] = matrix1[i][j] - matrix2[i][j];
38 | 38 |             }
39 | 39 |         }
40 | 40 |
41 | 41 |         return result;
42 | 42 |     }
43 | 43 |
44 | 44 |     public int[][] multiply(int[][] matrix1, int[][] matrix2) {
45 | 45 |         if (matrix1[0].length != matrix2.length) {
46 | 46 |             throw new IllegalArgumentException("Invalid matrix dimensions for multiplication");
47 | 47 |         }
48 | 48 |
49 | 49 |         int rows = matrix1.length;
```

Match #5: N-Gram Match Group

Similarity: 88.2%

BinarySearch: BinarySearch.java (Lines 4–17)

```
4 | 4 |  * Binary Search implementation
5 | 5 |  */
6 | 6 | public class BinarySearch {
7 | 7 |
8 | 8 |     public int search(int[] array, int target) {
9 | 9 |         if (array == null || array.length == 0) {
10 | 10 |             return -1;
11 | 11 |         }
12 | 12 |         return binarySearch(array, target, 0, array.length - 1);
13 | 13 |     }
14 | 14 |
15 | 15 |     private int binarySearch(int[] array, int target, int left, int right) {
16 | 16 |         if (left > right) {
17 | 17 |             return -1;
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 4–17)

```
4 | 4 |  * Binary Search - renamed version
5 | 5 |  */
6 | 6 | public class BinarySearch_Plagiarized1 {
7 | 7 |
8 | 8 |     public int find(int[] arr, int value) {
9 | 9 |         if (arr == null || arr.length == 0) {
10 | 10 |             return -1;
11 | 11 |         }
12 | 12 |         return binSearch(arr, value, 0, arr.length - 1);
13 | 13 |     }
14 | 14 |
15 | 15 |     private int binSearch(int[] arr, int value, int start, int end) {
16 | 16 |         if (start > end) {
17 | 17 |             return -1;
```

Match #6: N-Gram Match Group

Similarity: 81.8%

BinarySearch: BinarySearch.java (Lines 8–20)

```
8   8 |     public int search(int[] array, int target) {
9   9 |         if (array == null || array.length == 0) {
10  10 |             return -1;
11  11 |         }
12  12 |         return binarySearch(array, target, 0, array.length - 1);
13  13 |     }
14  14 |
15  15 |     private int binarySearch(int[] array, int target, int left, int right) {
16  16 |         if (left > right) {
17  17 |             return -1;
18  18 |         }
19  19 |
20  20 |         int mid = left + (right - left) / 2;
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 8–20)

```
8   8 |     public int find(int[] arr, int value) {
9   9 |         if (arr == null || arr.length == 0) {
10  10 |             return -1;
11  11 |         }
12  12 |         return binSearch(arr, value, 0, arr.length - 1);
13  13 |     }
14  14 |
15  15 |     private int binSearch(int[] arr, int value, int start, int end) {
16  16 |         if (start > end) {
17  17 |             return -1;
18  18 |         }
19  19 |
20  20 |         int middle = start + (end - start) / 2;
```

Match #7: N-Gram Match Group

Similarity: 81.8%

BinarySearch: BinarySearch.java (Lines 51–63)

```
51  51 |                 left = mid + 1;
52  52 |             }
53  53 |         }
54  54 |
55  55 |         return -1;
56  56 |     }
57  57 |
58  58 |     public int findFirst(int[] array, int target) {
59  59 |         int result = -1;
60  60 |         int left = 0;
61  61 |         int right = array.length - 1;
62  62 |
63  63 |         while (left <= right) {
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 25–38)

```
25  25 |
26  26 |         if (arr[middle] > value) {
27  27 |             return binSearch(arr, value, start, middle - 1);
28  28 |         } else {
29  29 |             return binSearch(arr, value, middle + 1, end);
30  30 |         }
31  31 |     }
32  32 |
33  33 |     public int findIterative(int[] arr, int value) {
34  34 |         if (arr == null || arr.length == 0) {
35  35 |             return -1;
36  36 |         }
37  37 |
38  38 |         int start = 0;
```

Match #8: N-Gram Match Group

Similarity: 85.7%

BinarySearch: BinarySearch.java (Lines 29–43)

```
29  29 |             return binarySearch(array, target, mid + 1, right);
30  30 |         }
31  31 |     }
32  32 |
33  33 |     public int searchIterative(int[] array, int target) {
34  34 |         if (array == null || array.length == 0) {
35  35 |             return -1;
36  36 |         }
37  37 |
38  38 |         int left = 0;
39  39 |         int right = array.length - 1;
40  40 |
41  41 |         while (left <= right) {
42  42 |             int mid = left + (right - left) / 2;
43  43 |
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 29–43)

```
29  29 |             return binSearch(arr, value, middle + 1, end);
30  30 |         }
31  31 |     }
32  32 |
33  33 |     public int findIterative(int[] arr, int value) {
34  34 |         if (arr == null || arr.length == 0) {
35  35 |             return -1;
36  36 |         }
37  37 |
38  38 |         int start = 0;
39  39 |         int end = arr.length - 1;
40  40 |
41  41 |         while (start <= end) {
42  42 |             int middle = start + (end - start) / 2;
43  43 |
```

Match #9: N-Gram Match Group

Similarity: 83.3%

BinarySearch: BinarySearch.java (Lines 47–63)

```
47 |
48 |             if (array[mid] > target) {
49 |                 right = mid - 1;
50 |             } else {
51 |                 left = mid + 1;
52 |             }
53 |         }
54 |
55 |         return -1;
56 |     }
57 |
58 |     public int findFirst(int[] array, int target) {
59 |         int result = -1;
60 |         int left = 0;
61 |         int right = array.length - 1;
62 |
63 |         while (left <= right) {
```

BinarySearch_Plagiarized1: BinarySearch_Plagiarized1.java (Lines 47–63)

```
47 |
48 |             if (arr[middle] > value) {
49 |                 end = middle - 1;
50 |             } else {
51 |                 start = middle + 1;
52 |             }
53 |         }
54 |
55 |         return -1;
56 |     }
57 |
58 |     public int locateFirst(int[] arr, int value) {
59 |         int res = -1;
60 |         int start = 0;
61 |         int end = arr.length - 1;
62 |
63 |         while (start <= end) {
```

Match #10: N-Gram Match Group

Similarity: 84.6%

BinarySearch: BinarySearch.java (Lines 33–44)

```
33 |     public int searchIterative(int[] array, int target) {
34 |         if (array == null || array.length == 0) {
35 |             return -1;
36 |         }
37 |
38 |         int left = 0;
39 |         int right = array.length - 1;
40 |
41 |         while (left <= right) {
42 |             int mid = left + (right - left) / 2;
43 |
44 |             if (array[mid] == target) {
```

StringUtils: StringUtils.java (Lines 9–20)

```
9 |         if (str == null || str.length() == 0) {
10 |             return true;
11 |         }
12 |
13 |         String cleaned = str.toLowerCase().replaceAll("[^a-z0-9]", "");
14 |         int left = 0;
15 |         int right = cleaned.length() - 1;
16 |
17 |         while (left < right) {
18 |             if (cleaned.charAt(left) != cleaned.charAt(right)) {
19 |                 return false;
20 |             }
```